

4.To override super class method with sub class method either super class method or subclass method as static then compiler will rise an error. If we declare both super and sub class method as static in method overriding compiler will not rise any error, JVM will provide output from the super class method.

NOTE: If we are trying to override super class static method with sub class static method then super class static method will override subclass static method, where JVM will generate output from super class static method.

EX:

```
1) class A {
2)     static void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) class B extends A {
7)     static void m1() {
8)         System.out.println("m1-B");
9)     }
10) }
11) class Test {
12)     public static void main(String args[]) {
13)         A a = new B();
14)         a.m1();
15)     }
16) }
```

5.To override super class method with subclass method, sub class method must have either same scope of the super class method or more scope when compared with super class method scope otherwise compiler will rise an error.

EX:

```
1) class A {
2)     protected void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) class B extends A {
7)     public void m1() {
8)         System.out.println("m1-B");
9)     }
10) }
11) class Test {
12)     public static void main(String args[]) {
13)         A a = new A();
```

```
14)    a.m1();
15) }
16)}
```

6.To override super class method with subclass method subclass method should have either same access privileges or weaker access privileges when compared with super class method access privileges.

Q)What are the differences between *Method Overloading* and *Method Overriding*?

- 1) The process of extending the existed method functionality with new functionality is called as **Method Overloading**.
The process of replacing existed method functionality with new functionality is called as **Method Overriding**.
- 2) In the case of method overloading, different method signatures must be provided to the methods
In the case of method overriding same method prototypes must be provided to the methods.
- 3) With OR without inheritance we can perform method overloading
With inheritance only we can perform Method overriding

EX:

```
1) class DB_Driver {
2)     public void getDriver() {
3)         System.out.println("Type-1 Driver");
4)     }
5) }
6) class New_DB_Driver extends DB_Driver {
7)     public void getDriver() {
8)         System.out.println("Type-4-Driver");
9)     }
10) }
11) class Test {
12)     public static void main(String args[]) {
13)         DB_Driver driver = new New_DB_Driver();
14)         driver.getDriver();
15)     }
16) }
```

In the above example, method overriding is implemented, in method overriding, for super class method call JVM has to execute subclass method, not super class method. In method overriding, always JVM is executing only subclass method, not super class method. In

method overriding, it is not suggestible to manage super class method body without execution, so that, we have to remove super class method body as part of code optimization. In Java applications, if we want to declare a method without body then we must declare that method as "Abstract Method".

If we want to declare abstract methods then the respective class must be abstract class.

```
1) abstract class DB_Driver {
2)     public abstract void getDriver();
3) }
4) class New_DB_Driver extends DB_Driver {
5)     public void getDriver() {
6)         System.out.println("Type-4 Driver");
7)     }
8) }
9) class Test {
10)     public static void main(String args[]) {
11)         DB_Driver driver = new New_DB_Driver();
12)         driver.getDriver();
13)     }
14) }
```

In Java applications, if we declare any abstract class with abstract methods, then it is convention to implement all the abstract methods by taking sub class.

To access the abstract class members, we have to create object for subclass and we have to create reference variable either for abstract class or for subclass.

If we create reference variable for abstract class then we are able to access only abstract class members, we are unable to access subclass own members

If we declare reference variable for subclass then we are able to access both abstract class members and subclass members.

```
1) abstract class A {
2)     void m1() {
3)         System.out.println("m1-A");
4)     }
5)     abstract void m2();
6)     abstract void m3();
7) }
8) class B extends A {
9)     void m2() {
10)         System.out.println("m2-B");
11)     }
12)     void m3() {
13)         System.out.println("m3-B");
14)     }
15)     void m4() {
```

```

16)      System.out.println("m4-B");
17)  }
18) }
19) class Test {
20)     public static void main(String args[]) {
21)         A a = new B();
22)         a.m1();
23)         a.m2();
24)         a.m3();
25)         //a.m4();---error
26)         B b = new B();
27)         b.m1();
28)         b.m2();
29)         b.m3();
30)         b.m4();
31)     }
32) }

```

In Java applications, it is not possible to create Object from abstract classes but it is possible to provide constructors in abstract classes, because, to recognize abstract class instance variables in order to store in the sub class objects.

EX:

```

1)  abstract class A {
2)      A() {
3)          System.out.println("A-Con");
4)      }
5)  }
6)  class B extends A {
7)      B() {
8)          System.out.println("B-Con");
9)      }
10) }
11) class Test {
12)     public static void main(String[] args) {
13)         B b = new B();
14)     }
15) }

```

In Java applications, if we declare any abstract class with abstract methods then it is mandatory to implement all the abstract methods in the respective subclass. If we implement only some of the abstract methods in the respective subclass then compiler will rise an error, where to come out from compilation error we have to declare the respective subclass as an abstract class and we have to provide implementation for the remaining abstract methods by taking another subclass in multilevel inheritance.

EX:

```
1) abstract class A {
2)     abstract void m1();
3)     abstract void m2();
4)     abstract void m3();
5) }
6) abstract class B extends A {
7)     void m1() {
8)         System.out.println("m1-A");
9)     }
10) }
11) class C extends B {
12)     void m2() {
13)         System.out.println("m2-C");
14)     }
15)     void m3() {
16)         System.out.println("m3-C");
17)     }
18) }
19) class Test {
20)     public static void main(String[] args) {
21)         A a = new C();
22)         a.m1();
23)         a.m2();
24)         a.m3();
25)     }
26) }
```

In Java applications, if we want to declare an abstract class then it is not at all mandatory condition to have at least one abstract method, it is possible to declare abstract class without having abstract methods but if we want to declare a method as an abstract method then the respective class must be abstract class.

```
1) abstract class A {
2)     void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) class B extends A {
7)     void m2() {
8)         System.out.println("m2-B");
9)     }
10) }
11) class Test {
12)     public static void main(String args[]) {
```

```

13)   A a = new B();
14)   a.m1();
15)   //a.m2();----->Error
16)   B b = new B();
17)   b.m1();
18)   b.m2();
19)   }
20) }

```

In Java applications, it is possible to extend an abstract class to concrete class and from concrete class to abstract class.

```

1) class A {
2)     void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) abstract class B extends A {
7)     abstract void m2();
8) }
9) class C extends B {
10)     void m2() {
11)         System.out.println("m2-C");
12)     }
13)     void m3() {
14)         System.out.println("m3-C");
15)     }
16) }
17) class Test {
18)     public static void main(String args[]) {
19)         A a = new C();
20)         a.m1();
21)         //a.m2();--->error
22)         //a.m3();--->error
23)         B b = new C();
24)         b.m1();
25)         b.m2();
26)         //b.m3();--->error
27)         C c = new C();
28)         c.m1();
29)         c.m2();
30)         c.m3();
31)     }
32) }

```

Note: In Java applications, it is not possible to extend a class to the same class, if we do the same then compiler will rise an error like "cyclic inheritance involving".

```
class A extends A{  
}
```

Status: Compilation Error: "cyclic inheritance involving".

```
class A extends B{  
}  
class B extends A{  
}
```

Status: Compilation Error: "cyclic inheritance involving".

Interfaces:

Interface is a Java Feature, it will allow only abstract methods.

In Java applications, for interfaces, we are able to create only reference variables, we are unable to create objects.

In the case of interfaces, by default, all the variables are "public static final".

In the case of interfaces, by default, all the methods are "public and abstract".

In Java applications, constructors are possible in classes and abstract classes but constructors are not possible in interfaces.

Interfaces will provide more sharability in Java applications when compared with classes and abstract classes.

In Java applications, if we declare any interface with abstract methods then it is convention to declare an implementation class for the interface and it is convention to provide implementation for all the abstract methods in implementation class.

```
1) interface I {  
2)     void m1();  
3)     void m2();  
4)     void m3();  
5) }  
6) class A implements I {  
7)     public void m1() {  
8)         System.out.println("m1-A");  
9)     }  
10)    public void m2() {
```

```

11)      System.out.println("m2-A");
12)  }
13)  public void m3() {
14)      System.out.println("m3-A");
15)  }
16)  public void m4() {
17)      System.out.println("m4-A");
18)  }
19) }
20) class Test {
21)     public static void main(String args[]) {
22)         I i = new A();
23)         i.m1();
24)         i.m2();
25)         i.m3();
26)         //i.m4();----->error
27)         A a = new A();
28)         a.m1();
29)         a.m2();
30)         a.m3();
31)         a.m4();
32)     }
33) }

```

In Java applications, if we declare an interface with abstract methods then it is mandatory to provide implementation for all the abstract methods in the respective implementation class. In this context if we provide implementation for some of the abstract methods at the respective implementation class then compiler will rise an error, where to come out from the compilation error we have to declare the respective implementation class as an abstract class and we have to provide implementation for the remaining abstract methods by taking a sub class for the abstract class.

```

1) interface I {
2)     void m1();
3)     void m2();
4)     void m3();
5) }
6) abstract class A implements I {
7)     public void m1() {
8)         System.out.println("m1-A");
9)     }
10) }
11) class B extends A {
12)     public void m2() {
13)         System.out.println("m2-B");

```



```

14) }
15) public void m3() {
16)     System.out.println("m3-B");
17) }
18) }
19) class Test {
20)     public static void main(String args[]) {
21)         I i = new I();
22)         i.m1();
23)         i.m2();
24)         i.m3();
25)         A a = new B();
26)         a.m1();
27)         a.m2();
28)         a.m3();
29)         B b = new B();
30)         b.m1();
31)         b.m2();
32)         b.m3();
33)     }
34) }

```

In Java applications, it is not possible to extend more than one class to a single class but it is possible to extend more than one interface to a single interface.

EX:

```

1) interface I1{
2)     void m1();
3) }
4) interface I2 {
5)     void m2();
6) }
7) interface I3 extends I1, I2 {
8)     void m3();
9) }
10) class A implements I3 {
11)     public void m1() {
12)         System.out.println("m1-A");
13)     }
14)     public void m2() {
15)         System.out.println("m2-A");
16)     }
17)     public void m3() {
18)         System.out.println("m3-A");
19)     }

```

```

20) }
21) class Test {
22)     public static void main(String args[]) {
23)         I1 i1 = new A();
24)         i1.m1();
25)         I2 i2 = new A();
26)         i2.m2();
27)         I3 i3 = new A();
28)         i3.m1();
29)         i3.m2();
30)         i3.m3();
31)         A a = new A();
32)         a.m1();
33)         a.m2();
34)         a.m3();
35)     }
36) }

```

In Java applications, it is possible to implement more than one interface into a single implementation class.

```

1) interface I1 {
2)     void m1();
3) }
4) interface I2 {
5)     void m2();
6) }
7) interface I3 {
8)     void m3();
9) }
10) class A implements I1, I2, I3 {
11)     public void m1() {
12)         System.out.println("m1-A");
13)     }
14)     public void m2() {
15)         System.out.println("m2-A");
16)     }
17)     public void m3() {
18)         System.out.println("m3-A");
19)     }
20) }
21) class Test {
22)     public static void main(String args[]) {
23)         I1 i1 = new A();
24)         i1.m1();

```

```

25)    I2 i2 = new A();
26)    i2.m2();
27)    I3 i3=new A();
28)    i3.m3();
29)    A a = new A();
30)    a.m1();
31)    a.m2();
32)    a.m3();
33) }
34) }

```

Q) Fond Valid Syntaxes between classes, abstract classes and Interfaces from the following list of syntaxes?

- 1) class extends class --->Valid
- 2) class extends class,class--->Invalid
- 3) class extends abstract class--->Valid
- 4) class extends abstract class,class--->Invalid
- 5) class extends abstract class,abstract class--->Invalid
- 6) class extends interface--->Invalid
- 7) class implements interface--->Valid
- 8) class implements interface,interface--->Valid
- 9) class implements interface extends class--->Invalid
- 10) class implements interface extends abstract class--->Invalid
- 11) class extends class implements interface--->Valid
- 12) class extends abstract class implements interface--->Valid
- 13) abstract class extends class--->Valid
- 14) abstract class extends abstract class--->Valid
- 15) abstract class extends class,class--->Invalid
- 16) abstract class extends abstract class,abstract class--->Invalid
- 17) abstract class extends class,abstract class--->Invalid
- 18) abstract class extends interface--->Invalid
- 19) abstract class implements interface--->Valid
- 20) abstract class implements interface,interface--->Valid
- 21) abstract class extends class implements interface--->Valid
- 22) abstract class extends abstract class implements interface--->Valid
- 23) abstract class implements interface extends class-->Invalid
- 24) abstract class implements interface extends abstract class-->Invalid
- 25) interface extends interface -->Valid
- 26) interface extends interface,interface -->Valid
- 27) interface extends class -->Invalid
- 28) interface extends abstract class -->Invalid
- 29) interface implements interface -->Invalid

Marker Interfaces:

Marker Interface is an Interface, it will not include any abstract method and it will provide some abilities to the objects at runtime of our Java application.

EX: java.io.Serializable , java.lang.Cloneable

java.io.Serializable:

The process of separating the data from an object is called as Serialization.

The process of reconstructing an Object on the basis of data is called as Deserialization.

Serialization & Deserialization

Where java.io.Serializable interface is a marker interface, it was not declared any method but it will make eligible any object for Serialization and Deserialization.

java.lang.Cloneable:

The process of generating duplicate object is called as Object Cloning.

IN java applications, by default, all objects are not eligible for Object cloning, only the objects which implements java.lang.Cloneable interface are eligible for Object cloning.

JAVA 8 Features over Interfaces

- Default Methods in Interfaces
- Static Methods in Interfaces
- Functional Interfaces

Default Methods in Interfaces:

In general, if we declare abstract methods in an interface then we have to implement all that interface methods in more number of classes with variable implementation part.

In the above context, if we require any method implementation common to every implementation class with fixed implementation then we have to implement that method in the interface as default method.

To declare default methods in interfaces we have to use "default" keyword in method syntax like access modifier.

EX:

```
1) interface I {  
2)     default void m1() {  
3)         System.out.println("m1-A");  
4)     }  
5) }  
6) class A implements I {}  
7) class Test {  
8)     public static void main(String args[]) {  
9)         I i = new A();
```

```
10)         i.m1();
11)     }
12)}
```

NOTE: It is possible to provide more than one default methods within a single interface.

EX:

```
interface I {
    default void m1() {
        -----
    }
    default void m2() {
        -----
    }
}
```

1. In JAVA8, it is possible to override default methods in the implementation classes.

```
1) interface I {
2)     default void m1() {
3)         System.out.println("m1-A");
4)     }
5) }
6) class A implements I {
7)     public void m1() {
8)         System.out.println("m1-A");
9)     }
10) }
11) class Test {
12)     public static void main(String args[]) {
13)         I i = new A();
14)         i.m1();
15)     }
16) }
```

Static Methods in Interfaces:

Up to JAVA7 version, static methods are not possible in interfaces but from JAVA8 version static methods are possible in interfaces in order to improve sharability.

If we declare static methods in the interfaces then it is not required to declare any implementation class to access that static method, we can use directly interface name to access static method.

NOTE: If we declare static methods in an interface then they will not be available to the respective implementation classes, we have to access static methods by using only interface names not even by using interface reference variable

EX:

```
1) interface I {
2)     static void m1() {
3)         System.out.println("m1-1");
4)     }
5) }
6) class Test {
7)     public static void main(String args[]) {
8)         I.m1();
9)     }
10)}
```

Note:In JAVA8 version, interfaces will allow concrete methods along with either "static" keyword or "default" keyword.

Functional Interface:

If any Java interface allows only one abstract method then it is called as "Functional Interface".

To make any interface as Functional Interface then we have to use the following annotation just above of the interface.

@FunctionalInterface

EX:java.lang Runnable
java.lang.Comparable

NOTE: In Functional Interfaces we have to provide only one abstract method but we can provide any number of default methods and any number of static methods.

```
1) @Functional Interface
2) interface I {
3)     void m1();
4)     //void m2();---->error
5)     default void m3() {
6)         System.out.println("m3-I");
7)     }
8)     static void m4() {
9)         System.out.println("m4-I");
10)    }
11) class A implements I {
12)     public void m1() {
13)         System.out.println("m1-A");
```

```

14)     }
15) }
16) class Test {
17)     public static void main(String args[]) {
18)         I i=new A();
19)         i.m1();
20)         i.m3();
21)         //i.m4();--->error
22)         I.m4();
23)         //A.m4();--->error
24)     }
25) }

```

Instanceof Operator:

It is a boolean operator, it can be used to check whether the specified reference variable is representing the specified class object or not that is compatible or not.

`ref_Var instanceof Class_Name`

where `ref_Var` and `Class_Name` must be related otherwise compiler will rise an error like "incompatible types error".

If `ref_Var` class is same as the specified `Class_Name` then instanceof operator will return "true".

If `ref_Var` class is subclass to the specified `Class_Name` then instanceof operator will return "true".

If `ref_Var` class is super class to the specified `Class_Name` then instanceof operator will return "false".

EX:

```

1) class A {
2) }
3) class B extends A {
4) }
5) class C {
6) }
7) class Test {
8)     public static void main(String args[]) {
9)         A a = new A();
10)        B b = new B();
11)        System.out.println(a instanceof A);
12)        System.out.println(a instanceof B);

```

```

13)      System.out.println(b instanceof A);
14)      //System.out.println(a instanceof C);
15)  }
16) }

```

Object Cloning:

The process of creating duplicate object for an existed object is called as Object Cloning.

If we want to perform Object Cloning in Java application then we have to use the following steps.

- 1) Declare an User defined Class.
- 2) Implement java.lang.Cloneable interface in order to make eligible any object for cloning.
- 3) Override Object class clone() method in user defined class.
public Object clone() throws CloneNotSupportedException
- 4) In Main class, in main() method, access clone() method over the respective object.

EX:

```

1) class Student implements Cloneable {
2)     String sid;
3)     String sname;
4)     String saddr;
5)     Student(String sid, String sname, String saddr) {
6)         this.sid = sid;
7)         this.sname = sname;
8)         this.saddr = saddr;
9)     }
10)    public Object clone() throws CloneNotSupportedException {
11)        return super.clone();
12)    }
13)    public String toString() {
14)        System.out.println("Student details");
15)        System.out.println("-----");
16)        System.out.println("Student Id :"+sid);
17)        System.out.println("Student name:"+sname);
18)        System.out.println("Student Address:"+saddr);
19)        return "";
20)    }
21) }
22) class Test {
23)     public static void main(String args[]) {
24)         Student std1 = new Student("S-111", "Durga", "Hyd");

```



```

25)      System.out.println("Student Details Before Cloning");
26)      System.out.println(std1);
27)
28)      Student std2 = (Student)std1.clone();
29)      System.out.println();
30)      System.out.println("Student Details After cloning");
31)      System.out.println(std2);
32)  }
33) }

```

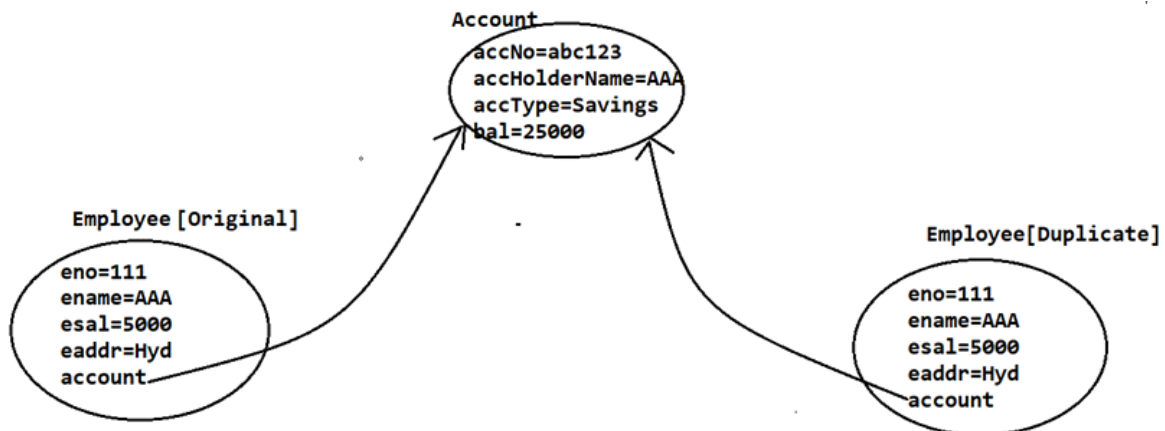
There are 2 types clonings in Java:

- Shallow Cloning/Shallow Copy
- Deep Cloning/Deep Copy

Shallow Cloning/Shallow Copy:

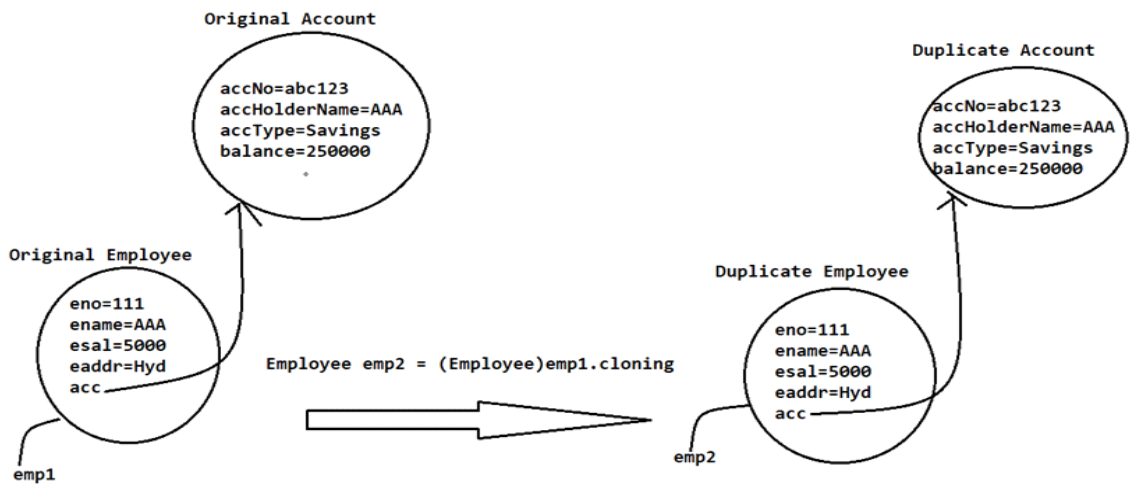
In this cloning mechanism, while cloning an object if any associated object is encountered then JVM will not duplicate associated object along with data duplication, where duplicated object is also refer the same associated object which was refereed by original object.

NOTE: Shallow Cloning is default cloning mechanism in Java.



Deep Cloning/Deep Copy:

In this cloning mechanism, while cloning an object if JVM encounter any associated object then JVM will duplicate associated object also along with data duplication. In this cloning mechanism, both original object and cloned object are having their own duplicated associated objects copy, both are not referring a single associated object.



EX:

```

1) class Account {
2)     String accNo;
3)     String accName;
4)     String accType;
5)     Account(String accNo, String accName, String accType) {
6)         this.accNo = accNo;
7)         this.accName = accName;
8)         this.accType = accType;
9)     }
10) }
11) class Employee implements Cloneable {
12)     String eid;
13)     String ename;
14)     String eddr;
15)     Account acc;
16)     Employee(String eid, String ename, String eaddr, Account acc) {
17)         this.eid = eid;
18)         this.ename = ename;
19)         this.eaddr = eaddr;
20)         this.acc = acc;
21)     }
22)     public Object clone() throws CloneNotSuppoertException {
23)         return super.clone();
24)     }
25)     public String toString() {
26)         System.out.println("Employee Details");
27)         System.out.println("-----");
28)         System.out.println("Employee Id :"+eid);
29)         System.out.println("Employee Address :"+eaddr);
30)         System.out.println("Employee Name :"+ename);

```

```

31)         System.out.println("Account Deatails");
32)         System.out.println("-----");
33)         System.out.println("Account Number:"+acc.accNo);
34)         System.out.println("Account Name :"+acc.accName);
35)         System.out.println("Account Type :"+acc.accType);
36)         System.out.println("Account Reference: "+acc);
37)         return "";
38)     }
39) }
40) class Test {
41)     public static void main(String args[]) throws Exception {
42)         Account acc = new Account("abc123", "Durga", "Savings");
43)         Employee emp1 = new Employee("E-111", "Durga", "Hyd", acc);
44)         System.out.println("Employee Details Before Cloning");
45)         System.out.println(emp1);
46)         Employee emp2 = (Employee)emp1.clone();
47)         System.out.println("Employee Details After Cloning");
48)         System.out.println(emp2);
49)     }
50) }

```

To prepare example for deep cloning provide the following clone() method in employee class in the above example of (shallow Cloning)

```

public Object clone()throws CloneNotSupportedException{
Account acc1=new Account(acc.accNo,acc.accName,acc.accType);
Employee emp=new Employee(eid,ename,eaddr,acc1);
return emp;
}

```

Adapter Classes:

In general, in Java applications when we implements an interface in any class then we must implement all the abstract methods of that interface with or without the requirement, it may increase unnecessary methods implementation in Java applications.

To overcome the above problem we have to use “Adapter classes“.

```

interface I{
    void m1();
    -----
    void m50();
}

```

```

abstract class M implements I {
    void m1(){ }
    -----
    abstract void m25();
    -----
    void m50(){ }
}

```

Adapter Class
Generic Class

```

class C1 implements I {
    void m1(){ ---- }
    void m25(){ --- }
}

```

```

class C2 implements I {
    void m2(){ --- }
    void m25(){ --- }
}

```

```

class C10 implements I {
    void m10(){ ---- }
    void m25(){ --- }
}

```

```

interface I{
    void m1();
    -----
    void m50();
}

```

```

abstract class M implements I {
    void m1(){ }
    -----
    abstract void m25();
    -----
    void m50(){ }
}

```

Adapter Class
Generic Class

```

class C1 implements I {
    void m1(){ ---- }
    void m25(){ --- }
}

```

```

class C2 implements I {
    void m2(){ --- }
    void m25(){ --- }
}

```

```

class C10 implements I {
    void m10(){ ---- }
    void m25(){ --- }
}

```

```
interface I{
    void m1();
    -----
    void m50();
}
```

```
abstract class M implements I {
    void m1(){ }
    -----
    abstract void m25();
    -----
    void m50(){ }
}
```

Adapter Class
Generic Class

~~extends M~~
class C1 ~~implements I~~{
 void m1(){ ---- }
 void m25(){ --- }
}

~~extends M~~
class C2 ~~implements I~~{
 void m2(){ --- }
 void m25(){ --- }
}

~~extends M~~
class C10 ~~implements I~~{
 void m10(){ ---- }
 void m25(){ --- }
}

```
interface I{
    void m1();
    -----
    void m50();
}
```

Adapter Class

```
abstract class A implements I{
    void m1(){ }
    -----
    abstract void m25();
    -----
    void m50(){ }
}
```

~~extends A~~
class C1 ~~implements I~~{
 void m1(){ --- }
 void m25(){ --- }
}

~~extends A~~
class C2 ~~implements I~~{
 void m2(){ --- }
 void m25(){ --- }
}

~~extends A~~
class C10 ~~implements I~~{
 void m10(){ --- }
 void m25(){ --- }
}

The main intention of Adapter classes is to reduce unnecessary methods implementations while implementing interfaces in implementation classes.

EX:

WindowAdapter

MouseAdapter

KeyAdapter

GenericServlet

ServletRequestWrapper

ServletResponseWrapper

HttpServletRequestWrapper

HttpServletResponseWrapper

```
interface I{
    void m1();
    -----
    void m50();
}
```

```
abstract class M implements I {
    void m1(){ }
    -----
    abstract void m25();
    -----
    void m50(){ }
}
```

Adapter Class
Generic Class

```
class C1 implements I {
    void m1(){ ---- }
    void m25(){ --- }
}
```

```
class C2 implements I {
    void m2(){ --- }
    void m25(){ --- }
}
```

```
class C10 implements I {
    void m10(){ ---- }
    void m25(){ --- }
}
```

Inner Classes

Inner Classes

Declaring a class inside a class is called as Inner class.

Syntax:

```
class Outer_Class
{
    class Inner_Class
    {
    }
}
```

In Java applications, Inner classes are able to provide the following advantages.

- 1) Modularity
- 2) Abstraction
- 3) Security
- 4) Sharability
- 5) Reusability

1) Modularity:

- In Java application, we are able to improve we modularity by using packages.
- If we want to divide our implementation in a class in the form of modules then we have to use Inner classes.

```
EX: class Account {
    class SavingsAccount{
        ----
    }
    class CurrentAccount {
        ----
    }
}
```

2) Abstraction:

If we declare any variable or method in an inner class then that variable and method are having scope up to that inner class only, it is not available to other inner classes, so that, Inner classes are able to improve Abstraction.

```
EX: class A {
    class B {
        int i = 10;
    }
    class C {
```



```

        void m1() {
            i = i+10; --> Error
        }
    }
}

```

3) Security:

It is not possible to declare a class as private, but, it is possible to declare an inner class as private, so that, inner classes are able to improve Security.

EX: private class A {
 }
 Status: Invalid

EX: class A {
 private class B {

 }
 }
 Status: Valid

4) Sharability:

It is not possible to declare a class as Static, but, it is possible to declare an inner class as static, so that, inner classes are able to improve Sharability.

EX: static class A {
 }
 Status: Invalid

EX: class A {
 static class B {
 }
 }
 Status: Valid

5) Reusability:

- In Java, inheritance feature will improve Code Reusability.
- In java applications, we can extend one inner class to another class, so that, inner classes are able to improve Code Reusability.

```
EX: class A {
    class B {
        ---
    }
    class C extends B {
        ---
    }
}
Status: Valid
```

There are four types of inner classes in java.

- Member Inner class
- Static Inner class
- Method Local Inner class
- Anonymous Inner class

Note: If we compile java file contains inner classes then Compiler will generate a separate .class for outer class and a separate .class file will be generated for inner class.

Outer_Class.class ---> for Outer class

Outer_Class\$Inner_Class.class---> For Inner class

□ **Member Inner Class:**

Declaring a normal class [Non static class] inside a class is called as member inner class.

```
EX: class Outer {
    class Inner {
        ----
    }
}
```

If we want to access the members of inner class then we have to create object for the inner class, for this, we have to use the following syntax.

```
Outer.Inner ref_Var = new Outer().new Inner();
```

- By using outer class reference variable we are able to access only outer class members, we are unable to access inner class members.
- By using Inner class reference variable we are Able to access only inner class members we are unable to access outer class members.
- By default, all outer class members are available to inner class, so we can access outer class members inside the inner class, but, Inner class members are not available to outer classes, we are unable to access inner class members in outer classes.

- Member inner classes are not allowing static declarations directly, but, static keyword is allowed along with final keyword.

EX:

```
1) class A
2) {
3)     void m1()
4)     {
5)         System.out.println("m1-A");
6)     }
7)     class B
8)     {
9)         //static int i=10;--> Error
10)        static final int j=20;
11)        void m2()
12)        {
13)            System.out.println("m2-B");
14)            System.out.println(j);
15)        }
16)        void m3()
17)        {
18)            System.out.println("m3-B");
19)        }
20)    }
21) }
22) class Test {
23)     public static void main(String[] args)
24)     {
25)         A a=new A();
26)         a.m1();
27)         //a.m2();--> Error
28)         A.B ab=new A().new B();
29)         ab.m2();
30)         ab.m3();
31)         //ab.m1();--> Error
32)     }
33) }
```

In Member inner classes, we can extend one inner class to another inner class but both the inner classes must be provided with in the same outer class.

EX:

```
1) class A
2) {
3)     class B
4)     {
5)         void m1()
6)         {
7)             System.out.println("m1-B");
8)         }
9)         void m2()
10)        {
11)            System.out.println("m2-B");
12)        }
13)    }
14)    class C extends B
15)    {
16)        void m3()
17)        {
18)            System.out.println("m3-C");
19)        }
20)        void m4()
21)        {
22)            System.out.println("m4-C");
23)        }
24)    }
25) }
26) class Test
27) {
28)     public static void main(String[] args)
29)     {
30)         A.B ab=new A().new C();
31)         ab.m1();
32)         ab.m2();
33)         //ab.m3();--> Error
34)         A.C ac=new A().new C();
35)         ac.m1();
36)         ac.m2();
37)         ac.m3();
38)         ac.m4();
39)     }
40) }
```

We cannot extend one inner class to another inner class which are available in two different outer classes.

EX:

```
class A {  
    class B {  
        ---  
    }  
}  
class C {  
    class D extends A.B {  
        ---  
    }  
}
```

Status: Invalid.

We can extend an inner class from an outer class.

EX:

```
class A {  
    ---  
}  
class B {  
    class C extends A {  
        ----  
    }  
}
```

Status: Valid

We can extend the immediate outer class to its inner class.

EX:

```
class A {  
    class B extends A {  
        ---  
    }  
}
```

Q) Is it possible to write an interface inside a class?

Yes, it is possible to provide an interface inside a class, but, the respective implementation class must be provided with in the same outer class.

EX:

```
1) class A
2) {
3)     interface I
4)     {
5)         void m1();
6)         void m2();
7)         void m3();
8)     }
9)     class B implements I
10)    {
11)        public void m1()
12)        {
13)            System.out.println("m1-B");
14)        }
15)        public void m2()
16)        {
17)            System.out.println("m2-B");
18)        }
19)        public void m3()
20)        {
21)            System.out.println("m3-B");
22)        }
23)    }
24) }
25) class Test
26) {
27)     public static void main(String[] args)
28)     {
29)         A.I ai=new A().new B();
30)         ai.m1();
31)         ai.m2();
32)         ai.m3();
33)     }
34) }
```

EX:

```
1) class A
2) {
3)     abstract class B
4)     {
5)         void m1()
6)         {
7)             System.out.println("m1-B");
8)         }
```

```

9)     abstract void m2();
10)    abstract void m3();
11)    }
12)    class C extends B
13)    {
14)        void m2()
15)        {
16)            System.out.println("m2-C");
17)        }
18)        void m3()
19)        {
20)            System.out.println("m3-C");
21)        }
22)    }
23) }
24) class Test
25) {
26)     public static void main(String[] args)
27)     {
28)         A.B ab=new A().new C();
29)         ab.m1();
30)         ab.m2();
31)         ab.m3();
32)     }
33) }

```

EX:

```

1) package com.durgasoft.core;
2)
3) abstract class A{
4)     class B{
5)         void m1(){
6)             System.out.println("m1-B");
7)         }
8)         void m2(){
9)             System.out.println("m2-B");
10)        }
11)        void m3(){
12)            System.out.println("m3-B");
13)        }
14)    }
15) }
16) class C extends A{
17)     // class B{ }
18) }

```

```

19) class Test{
20)     public static void main(String[] args){
21)         A.B ab = new C().new B();
22)         ab.m1();
23)         ab.m2();
24)         ab.m3();
25)     }
26) }

```

EX:

```

1) package com.durgasoft.core;
2)
3) abstract class A{
4)     abstract class B{
5)         void m1(){
6)             System.out.println("m1-B");
7)         }
8)         abstract void m2();
9)         abstract void m3();
10)    }
11) class C extends B{
12)     @Override
13)     void m2() {
14)         System.out.println("m2-C");
15)     }
16)
17)     @Override
18)     void m3() {
19)         System.out.println("m3-C");
20)     }
21) }
22) }
23) class D extends A{
24)
25) }
26) class Test{
27)     public static void main(String[] args){
28)         A.B ab = new D().new C();
29)         ab.m1();
30)         ab.m2();
31)         ab.m3();
32)     }
33) }

```


Ex:

```
1) package com.durgasoft.core;
2)
3) abstract class A{
4)     interface I{
5)         void m1();
6)         void m2();
7)         void m3();
8)     }
9)     class B implements I{
10)         @Override
11)         public void m1() {
12)             System.out.println("m1-B");
13)         }
14)
15)         @Override
16)         public void m2() {
17)             System.out.println("m2-B");
18)         }
19)
20)         @Override
21)         public void m3() {
22)             System.out.println("m3-B");
23)         }
24)     }
25) }
26) class C extends A{
27)
28) }
29) class Test{
30)     public static void main(String[] args){
31)         A.I ai = new C().new B();
32)         ai.m1();
33)         ai.m2();
34)         ai.m3();
35)     }
36) }
```

□ Static Inner Class:

Declaring a static class inside a class is called as Static inner class.

Syntax:

```
class Outer
{
    static class Inner
    {
        ----
    }
}
```

If we want to access the members of static inner class we have to create object for static inner class, for this we have to use the following syntax.

```
Outer.Inner ref_Var = new Outer.Inner();
```

- Static inner classes are able to allow only static members of the outer class, it will not allow non static members of the outer class.
- In general, inner classes are not allowing static keyword directly, but, static inner class is able to allow static declarations directly.

EX:

```
1) class A
2) {
3)     static class B
4)     {
5)         void m1()
6)         {
7)             System.out.println("m1-B");
8)         }
9)         void m2()
10)        {
11)            System.out.println("m2-B");
12)        }
13)        static void m3()
14)        {
15)            System.out.println("m3-B");
16)        }
17)    }
18) }
19) class Test
20) {
21)     public static void main(String[] args)
```

```
22) {  
23)     A.B ab=new A.B();  
24)     ab.m1();  
25)     ab.m2();  
26)     A.B.m3();  
27) }  
28) }
```

Q) Is it possible to write a class inside an interface?

Yes, it is possible to write a class inside an interface, If we declare a class inside an interface then that class is converted as a static inner class, we can access members of this inner class like static inner class members.

EX:

```
1) interface I  
2) {  
3)     class A // static class A  
4)     {  
5)         void m1()  
6)         {  
7)             System.out.println("m1-A");  
8)         }  
9)         void m2()  
10)        {  
11)            System.out.println("m2-A");  
12)        }  
13)    }  
14) }  
15) class Test  
16) {  
17)     public static void main(String[] args)  
18)     {  
19)         I.A ia=new I.A();  
20)         ia.m1();  
21)         ia.m2();  
22)     }  
23) }
```

Note: We can write abstract class inside an interface, but, the respective sub class must be declared in the same interface, here the declared sub class is converted as static inner class.

EX:

```
1) interface I
2) {
3)     abstract class A
4)     {
5)         void m1()
6)         {
7)             System.out.println("m1-A");
8)         }
9)         abstract void m2();
10)        abstract void m3();
11)    }
12)    class B extends A
13)    {
14)        void m2()
15)        {
16)            System.out.println("m2-B");
17)        }
18)        void m3()
19)        {
20)            System.out.println("m3-B");
21)        }
22)    }
23) }
24) class Test
25) {
26)     public static void main(String[] args)
27)     {
28)         I.A ia=new I.B();
29)         ia.m1();
30)         ia.m2();
31)         ia.m3();
32)     }
33) }
```

NOTE: We can write an interface inside an interface, but, the respective implementation class must be provided with in the same outer interface.

EX:

```
1) interface I1
2) {
3)     interface I2
4)     {
5)         void m1();
6)         void m2();
```

```

7)      void m3();
8)  }
9)  class A implements I2
10) {
11)     public void m1()
12)     {
13)         System.out.println("m1-A");
14)     }
15)     public void m2()
16)     {
17)         System.out.println("m2-A");
18)     }
19)     public void m3()
20)     {
21)         System.out.println("m3-A");
22)     }
23) }
24) }
25) class Test
26) {
27)     public static void main(String[] args)
28)     {
29)         I1.I2 i12=new I1.A();
30)         i12.m1();
31)         i12.m2();
32)         i12.m3();
33)     }
34) }

```

□ Method Local Inner Class:

Declaring class inside a method is called as Method Inner class.

If we declare a class inside a method then the scope of that is available up to the respective method only, we have to create object for that inner class in the respective method only, we have to access the members of that inner class inside the respective method.

EX:

```

1) class A
2) {
3)     void m1()
4)     {
5)         class B
6)         {
7)             void m2()

```

```

8)      {
9)      System.out.println("m2-B");
10)     }
11)     void m3()
12)     {
13)         System.out.println("m3-B");
14)     }
15)     }//B
16)     B b=new B();
17)     b.m2();
18)     b.m3();
19) }//m1()
20) }//A
21) class Test
22) {
23)     public static void main(String[] args)
24)     {
25)         A a=new A();
26)         a.m1();
27)     }
28) }

```

□ Anonymous Inner Classes:

Anonymous inner classes are nameless inner classes, these are used to provide implementation for abstract classes and interfaces.

Note: For abstract class we may take sub classes and for interfaces we may take implementation classes to provide implementations , but, here sub classes of the abstract class and implementation class of the interface are able to allow their own members and these sub classes and implementation classes objects are having their identity, not interface identity and abstract class identity.

In java applications, if we want to provide implementations for only abstract class members or interface members and if we want to create objects with interface identity and with abstract class identity then we have to use Anonymous inner classes.

Syntax:

```

abstract class Name / interface Name
{
    ----
}
class Outer {
    Name ref_Var = new Name()

```

```

    {
        ---implementation----
    };
}

```

EX:

```

1) abstract class A
2) {
3)     void m1()
4)     {
5)         System.out.println("m1-A");
6)     }
7)     abstract void m2();
8)     abstract void m3();
9) }
10) class Outer
11) {
12)     A a=new A()
13)     {
14)         void m2()
15)         {
16)             System.out.println("m2-AIC");
17)         }
18)         void m3()
19)         {
20)             System.out.println("m3-AIC");
21)         }
22)         void m4()
23)         {
24)             System.out.println("m4-AIC");
25)         }
26)     };
27) }
28) class Test
29) {
30)     public static void main(String[] args)
31)     {
32)         Outer o=new Outer();
33)         o.a.m1();
34)         o.a.m2();
35)         o.a.m3();
36)         //o.a.m4();--> Error
37)     }
38) }

```

EX:

```
1) interface I
2) {
3)     void m1();
4)     void m2();
5)     void m3();
6) }
7) class Outer
8) {
9)     I i=new I()
10)    {
11)        public void m1()
12)        {
13)            System.out.println("m1-AIC");
14)        }
15)        public void m2()
16)        {
17)            System.out.println("m2-AIC");
18)        }
19)        public void m3()
20)        {
21)            System.out.println("m3-AIC");
22)        }
23)        public void m4()
24)        {
25)            System.out.println("m4-AIC");
26)        }
27)    };
28) }
29) class Test
30) {
31)     public static void main(String[] args)
32)     {
33)         Outer o=new Outer();
34)         o.i.m1();
35)         o.i.m2();
36)         o.i.m3();
37)         //o.i.m4();--> Error
38)     }
39) }
```

In general, we will use Anonymous Inner classes when we have requirement to pass any interface or abstract class references as parameters to the methods.

EX:

```
1) interface I
2) {
3)     void m1();
4)     void m2();
5)     void m3();
6) }
7) class A
8) {
9)     void meth(I i)
10)    {
11)        i.m1();
12)        i.m2();
13)        i.m3();
14)    }
15) }
16) class Test
17) {
18)     public static void main(String[] args)
19)     {
20)         A a=new A();
21)         a.meth(new I()
22)         {
23)             public void m1()
24)             {
25)                 System.out.println("m1-AIC");
26)             }
27)             public void m2()
28)             {
29)                 System.out.println("m2-AIC");
30)             }
31)             public void m3()
32)             {
33)                 System.out.println("m3-AIC");
34)             }
35)             public void m4()
36)             {
37)                 System.out.println("m4-AIC");
38)             }
39)         });
40)     }
41) }
```

WRAPPER CLASSES